

KITTI + RetinaNet Object Detection (Stable Submission Version)

핵심 수정:

- torchvision RetinaNet num_classes 정식 사용
- Normalize + Resize 추가
- KITTI label string/int 모두 대응 가능 구조
- 안정적인 evaluation (object-level IoU)
- num_workers 안전화

```
import os
import random
from typing import List, Dict, Tuple

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

import torchvision
from torchvision.models.detection import retinanet_resnet50_fpn
import torchvision.transforms.functional as F

import matplotlib.pyplot as plt
import matplotlib.patches as patches

# -----
# 1. Seed
# -----
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed(42)

# -----
# 2. KITTI Dataset (Stable)
# -----
class KITTIDataset(torch.utils.data.Dataset):
```

```

def __init__(self, root, split="train"):
    self.root = root
    split_file = os.path.join(root, "splits", f"{split}.txt")

    with open(split_file, "r") as f:
        self.ids = [x.strip() for x in f.readlines() if x.strip()]

    # class mapping (KITTI 기준 최소 버전)
    self.class_map = {
        "Car": 1,
        "Pedestrian": 2,
        "Cyclist": 3
    }

    print(f"[Dataset] {split} images: {len(self.ids)}")

def __len__(self):
    return len(self.ids)

def _load_image(self, img_id):
    path = os.path.join(self.root, "images", img_id + ".png")
    img = torchvision.io.read_image(path).float() / 255.0

    # Resize (stable training)
    img = F.resize(img, [512, 512])
    return img

def _load_target(self, img_id):
    label_path = os.path.join(self.root, "labels", img_id + ".txt")

    boxes = []
    labels = []

    if os.path.exists(label_path):
        with open(label_path, "r") as f:
            for line in f:
                parts = line.strip().split()
                if len(parts) != 5:
                    continue

                cls = parts[0]
                if cls not in self.class_map:

```

```

        continue

        xmin, ymin, xmax, ymax = map(float, parts[1:])

        boxes.append([xmin, ymin, xmax, ymax])
        labels.append(self.class_map[cls])

    boxes = torch.tensor(boxes, dtype=torch.float32)
    labels = torch.tensor(labels, dtype=torch.int64)

    return {"boxes": boxes, "labels": labels}

def __getitem__(self, idx):
    img_id = self.ids[idx]

    img = self._load_image(img_id)
    target = self._load_target(img_id)

    return img, target

# -----
# 3. Model
# -----
def get_model(num_classes):
    model = retinanet_resnet50_fpn(weights="DEFAULT")

    # torchvision 공식 방식 (중요)
    in_features = model.head.classification_head.num_classes

    # replace head properly
    model.head.classification_head.num_classes = num_classes
    model.head.classification_head.cls_logits = nn.Conv2d(
        model.backbone.out_channels,
        num_classes * model.head.classification_head.num_anchors,
        kernel_size=3,
        stride=1,
        padding=1
    )

    return model

# -----

```

4. Utils

```
def collate_fn(batch):  
    return tuple(zip(*batch))
```

```
def box_iou(a, b):  
    if len(a) == 0 or len(b) == 0:  
        return torch.zeros((len(a), len(b)))
```

```
    a = a[:, None, :]  
    b = b[None, :, :]
```

```
    inter_x1 = torch.max(a[:, :, 0], b[:, :, 0])  
    inter_y1 = torch.max(a[:, :, 1], b[:, :, 1])  
    inter_x2 = torch.min(a[:, :, 2], b[:, :, 2])  
    inter_y2 = torch.min(a[:, :, 3], b[:, :, 3])
```

```
    inter = (inter_x2 - inter_x1).clamp(0) * (inter_y2 - inter_y1).clamp(0)
```

```
    area_a = (a[:, :, 2] - a[:, :, 0]) * (a[:, :, 3] - a[:, :, 1])  
    area_b = (b[:, :, 2] - b[:, :, 0]) * (b[:, :, 3] - b[:, :, 1])
```

```
    union = area_a + area_b - inter
```

```
    return inter / (union + 1e-6)
```

5. Evaluation (stable)

```
@torch.no_grad()  
def evaluate(model, loader, device, thr=0.5):
```

```
    model.eval()
```

```
    correct = 0
```

```
    total = 0
```

```
    for imgs, targets in loader:  
        imgs = [img.to(device) for img in imgs]
```

```
        outputs = model(imgs)
```

```

for out, tgt in zip(outputs, targets):

    pred = out["boxes"].cpu()
    gt = tgt["boxes"]

    if len(gt) == 0:
        continue

    ious = box_iou(pred, gt)

    if (ious > thr).any():
        correct += 1

    total += 1

return correct / max(total, 1)

```

```

# -----
# 6. Visualization
# -----

```

```

def visualize(model, dataset, device, save_dir="vis"):

```

```

    os.makedirs(save_dir, exist_ok=True)
    model.eval()

```

```

    for i in range(5):

```

```

        img, tgt = dataset[random.randint(0, len(dataset)-1)]
        out = model([img.to(device)])[0]

```

```

        img_np = img.permute(1,2,0).cpu().numpy()

```

```

        fig, ax = plt.subplots(1)
        ax.imshow(img_np)

```

```

        for b in tgt["boxes"]:
            x1,y1,x2,y2 = b
            ax.add_patch(patches.Rectangle(
                (x1,y1), x2-x1, y2-y1,
                edgecolor="blue", facecolor="none"
            ))

```

```

        for b in out["boxes"][:10]:

```

```

x1,y1,x2,y2 = b.cpu0
ax.add_patch(patches.Rectangle(
    (x1,y1), x2-x1, y2-y1,
    edgecolor="red", facecolor="none"
))

```

```

plt.axis("off")
plt.savefig(f"{save_dir}/vis_{i}.png")
plt.close()

```

```

# -----
# 7. Train
# -----

```

```

def train(root):

```

```

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

```

    train_ds = KITTTIDataset(root, "train")

```

```

    val_ds = KITTTIDataset(root, "val")

```

```

    train_loader = DataLoader(train_ds, batch_size=2, shuffle=True, num_workers=0,
collate_fn=collate_fn)

```

```

    val_loader = DataLoader(val_ds, batch_size=2, shuffle=False, num_workers=0,
collate_fn=collate_fn)

```

```

    num_classes = 4 # background + 3 classes

```

```

    model = get_model(num_classes)

```

```

    model.to(device)

```

```

    optimizer = optim.SGD(model.parameters(), lr=0.003, momentum=0.9,
weight_decay=0.0005)

```

```

    best = 0

```

```

    for epoch in range(10):

```

```

        model.train()

```

```

        loss_sum = 0

```

```

        for imgs, targets in train_loader:

```

```

            imgs = [i.to(device) for i in imgs]

```

```

        targets = [{k: v.to(device) for k,v in t.items()} for t in targets]

        loss_dict = model(imgs, targets)
        loss = sum(loss_dict.values())

        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 10)
        optimizer.step()

        loss_sum += loss.item()

    acc = evaluate(model, val_loader, device)

    print(f"Epoch {epoch+1} loss={loss_sum:.4f} val_acc={acc:.4f}")

    if acc > best:
        best = acc
        torch.save(model.state_dict(), os.path.join(root, "best_retinanet.pth"))

return model

```

```

# -----
# 8. Main
# -----
def main():

```

```

    root = "/content/kitti"

    model = train(root)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    ds = KITTIIDataset(root, "val")

    visualize(model, ds, device)

```

```

if __name__ == "__main__":
    main()

```